

Microphone Fingerprinting Using Supervised Learning Algorithms

Raul-Florin Ciure
Faculty of Automation and Computers
Politehnica University of Timișoara
Timișoara, Romania
raul.ciure@student.upt.ro

Keywords — *fingerprinting, machine learning, classifiers, supervised learning algorithms*

I. INTRODUCTION

The ubiquity of mobile devices, in particular that of smartphones, has made them significantly integrated into critical aspects of daily life, ranging from financial transactions to multi-factor authentication. As these devices are in the process of becoming the primary tools for digital identity, there is a substantial need for robust security mechanisms. Traditional authentication methods typically rely on software-based identifiers – such as IMSI, MAC addresses, browser cookies, or DUIDs. While effective for general use, these software identifiers are fundamentally malleable: they can be spoofed, reset, or modified by malicious actors, leaving systems vulnerable to impersonation attacks [1].

To address these vulnerabilities, some security research has turned toward the concept of hardware fingerprinting. This technique relies on the physical reality that no two hardware components are perfectly identical due to microscopic differences originating from the manufacturing process. Thus, every sensor – whether it is an accelerometer, a gyroscope, or a microphone – possesses unique, non-clonable physical imperfections. These imperfections result in subtle deviations in performance that remain consistent over time, effectively creating a hardware-based identity for the device itself [2].

Among the various sensors available on modern smartphones, the microphone offers a particularly compelling way for fingerprinting. As an analog-to-digital component, a microphone consists, in a standard design, of a diaphragm, a backplate, and pre-amplification circuitry, all of them introducing unique distortions in the recorded audio. These slight distortions are most visible in the frequency response of the device.

Even when two phones of the exact same brand and model record the same sound, the specific sensitivity they exhibit at different frequencies will differ slightly. By analyzing the amplitudes of the power spectrum it is possible to isolate these unique hardware signatures [3].

This paper presents a method for performing microphone fingerprinting using supervised learning algorithms. The main objectives are to identify the source devices, based solely on the spectral characteristics of the audio they record, and to compare the accuracy and performance metrics of the different algorithms employed.

II. LEARNING ALGORITHMS

To identify which smartphone recorded a specific audio file, this project was treated as a standard classification problem. The set of features are represented by the power spectrum samples obtained from the audio recordings, with the labels represented by the phone unit or model.

Four different algorithms were selected for testing in order to determine which one works best for audio fingerprinting, in general, and for this specific dataset, in particular. The models employed were chosen because they represent the most common classes of supervised machine learning algorithms: *linear*, *ensemble-based*, *connectionist*, and *instance-based*. Below is a succinct description of the chosen models, each corresponding to a specific algorithm class mentioned previously.

A. Support Vector Machine (SVM)

The Support Vector Machine (SVM) is a discriminative classification algorithm formally defined by a separating hyperplane. In the context of high-dimensional spectral data, SVMs are particularly robust due to their formulation, which minimizes a bound on the generalization error rather than the empirical error.

While SVMs frequently employ non-linear kernels, such as the Radial Basis Function, to handle complex decision boundaries, this study utilizes a linear kernel. This design choice is based on the inherent characteristics of our feature space, which consists of 4,096 amplitude samples. In such a high-dimensional space, data points tend to be sparsely distributed, making them linearly separable with a high probability [4].

The adoption of a linear kernel offers two distinct advantages for this application. Firstly, it significantly reduces computational complexity at run time compared to non-linear kernels, as prediction requires only a dot product rather than the evaluation of kernel functions against all support vectors. Secondly, it imposes a stronger resistance against overfitting by restricting the decision boundary to a hyperplane. Thus the model is less likely to capture noise artifacts present in individual audio recordings, thereby improving the model accuracy.

B. Random Forest (RF)

The Random Forest algorithm is an ensemble learning algorithm that operates by constructing a multitude of decision trees at training time. It is intrinsically suited for acoustic fingerprinting due to its resilience to noise and its ability to model non-linear interactions between frequency bands without requiring feature scaling.

Each individual tree in the forest is trained on a random subset of the data sampled. Furthermore, at each candidate split in the learning process, only a random subset of features is considered. The final classification is derived via a majority voting mechanism across all trees in the ensemble. This aggregation effectively mitigates the risk of overfitting associated with individual decision trees. In the context of spectral data, this allows the model to identify robust frequency signatures while ignoring transient spectral anomalies that may affect only a subset of the trees [5].

C. Multi-Layer Perceptron (MLP)

The Multi-Layer Perceptron is a class of feedforward artificial neural network (i.e. connectionist) models that approximates the relationship between inputs and outputs through a series of non-linear transformations.

The architecture employed consists of an input layer matching the dimensionality of the feature vectors, followed by one or more hidden layers, and culminating in a softmax output layer that produces a probability distribution over the provided classes. The hidden layers utilize non-linear activation functions, thus allowing the MLP to learn complex, hierarchical representations of the data. Unlike linear models, the MLP can capture complex dependencies between distant components, effectively learning, in the specific context of this paper, a representation of each microphone's transfer function [6].

D. K-Nearest Neighbors (KNN)

This algorithm is a non-parametric, instance-based learning algorithm. In contrast to other types of learning algorithms that build an explicit model during the training phase, KNN is a so called "lazy" learner that postpones computation until classification is required.

The algorithm operates on the premise of feature similarity. As such, an unclassified sample is assigned to the class most common among its k-nearest neighbors in the feature space [7].

Although computationally intensive for high-dimensional data during the inference phase, KNN serves as a critical baseline for evaluating the topology of the feature space. A high performance result by a KNN classifier indicates that the intra-class variance is low and that class specific features form distinct, compact clusters within the dataset. Conversely, poor performance would suggest that the classes are not separable by simple geometric proximity.

III. TESTING METHODOLOGY AND DATASET

The methodology employed for data gathering with the purpose of comparing the four machine learning models is as follows. Three different test scenarios were conducted, each focusing on a specific fingerprinting context.

The first scenario (scenario A) aims to distinguish between recording made using different units of the same mobile phone model, trying to use the slight differences in the frequency response patterns for each different microphone, as resulting from the manufacturing process, to identify a specific unit corresponding to a given pattern, thus treating the microphone of each unit as having a corresponding physical unclonable function attached to it.

The second and third scenarios aim to distinguish between the recordings made with different phone models, from a

diverse group of manufacturers, using three types of environmental sounds. The second scenario (scenario B) uses clean recordings, i.e. live recordings with minimal noise, for training and testing while the third scenario (scenario C) uses recordings that have extra ambient noise, specifically music playing in the background.

For all the test scenarios the dataset used is the one compiled by A. Berdich et al. [3] for use in their paper which focuses on the same subject. The dataset contains the power spectrum for each recording, consisting of 4096 samples at ~5Hz resolution in the frequency range from 0Hz to 22050Hz. Based on the structure of the above dataset, this paper uses the synthetically reproduced environmental sounds for scenario A, the clean live recordings for scenario B, and the noisy live recordings for scenario C.

IV. TESTING AND RESULTS

Training and testing of the models was done using Python (CPython 3.14.2) and the *pandas* and *scikit-learn* libraries. The metrics that were taken into account for the comparison were *accuracy*, *precision*, *recall*, *F1-score*, *receiver operating characteristic area under curve (ROC AUC)*, *log loss* and the *confusion matrix*.

Relevant configuration options used in each model are as follows: for SVM a linear kernel was used, together with probability estimation; for RF the number of estimators was set to the default value of 100; for MLP the hidden layer size was set to the default, while the maximum number of iterations was set to 1000; for KNN the default value of 5 neighbours was used. For output reproducibility, all algorithms which support a predefined random seed had it set to value 42.

Separate tests were performed for each of the scenarios, with the results described below. A consolidated view of the results for all the scenarios is also shown in Table 1.

TABLE I. COMBINED PERFORMANCE METRICS FOR ALL SCENARIOS

Sce.	Alg.	Performance metrics (averages)					
		Acc.	Prec.	Rec.	F1	AUC	Log Loss
A	SVM	1.00	1.00	1.00	1.00	1.00	0.05
	RF	1.00	1.00	1.00	1.00	1.00	0.07
	MLP	0.99	0.99	0.99	0.99	1.00	0.08
	KNN	0.95	0.95	0.95	0.94	1.00	0.33
B	SVM	1.00	1.00	1.00	1.00	1.00	0.01
	RF	1.00	1.00	1.00	1.00	1.00	0.04
	MLP	1.00	1.00	1.00	1.00	1.00	0.04
	KNN	1.00	1.00	1.00	1.00	1.00	0.02
C	SVM	0.81	0.80	0.79	0.79	0.99	0.57
	RF	0.93	0.92	0.91	0.91	1.00	0.46
	MLP	0.86	0.85	0.83	0.82	1.00	0.46
	KNN	0.55	0.58	0.50	0.53	0.83	9.80

A. Scenario A

In this scenario, the recordings from 16 distinct units (labeled from A to P) of the same phone model (Samsung Galaxy S6) were used as the classification target.

The results, presented in Fig. 1, show a high level of performance, in terms of the metrics taken into account, for all of the four models, with SVM and RF performing a perfect classification, followed closely by MLP. The KNN model shows good performance, but visibly lower than the other three, while also having a significantly higher log loss value, meaning that it made wrong classifications with a high degree of confidence. These results indicate that the models used, especially SVM and Random Forest could accurately classify the different microphone units based on the supposed minimal differences caused by the manufacturing process. The confusion matrices for each model are shown in Fig. 2, Fig. 3, Fig. 4 and Fig. 5.

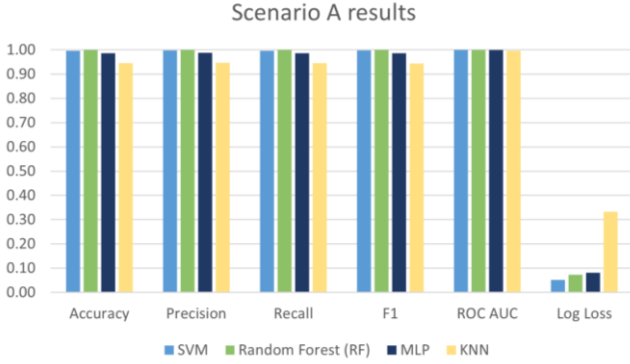


Fig. 1. Scenario A results chart

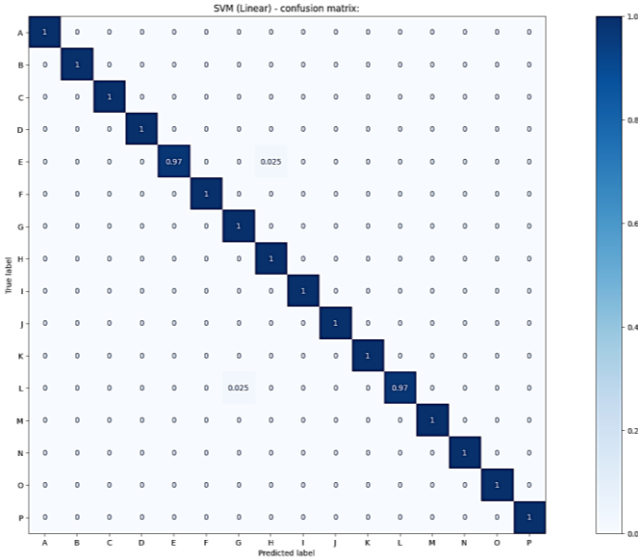


Fig. 2. Scenario A – SVM model confusion matrix

B. Scenario B

In this scenario, the recordings of clean environmental sounds from 18 phones, of different models and manufacturers, labeled accordingly, were used as the target for the classification.

The combined results, shown in Fig. 6, reveal that, in this specific test case, all the four algorithms performed perfectly, based on the considered metrics. This indicates that the hardware variability between the various device models is high enough to allow for a precise classification for each of the learning models used. The confusion matrices for each learning model are presented in Fig. 7, Fig. 8, Fig. 9 and Fig. 10.

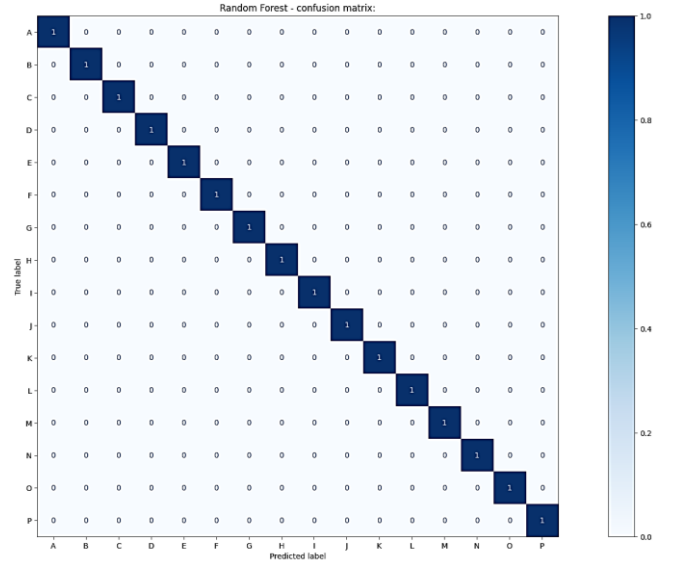


Fig. 3. Scenario A – RF model confusion matrix

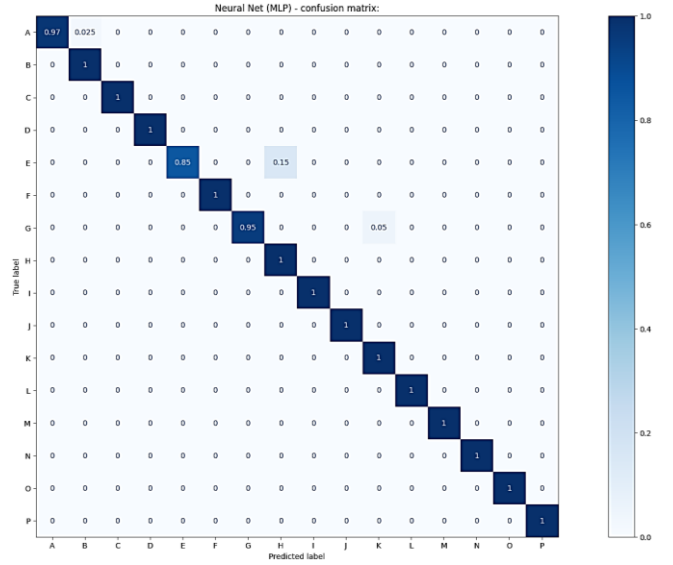


Fig. 4. Scenario A – MLP model confusion matrix

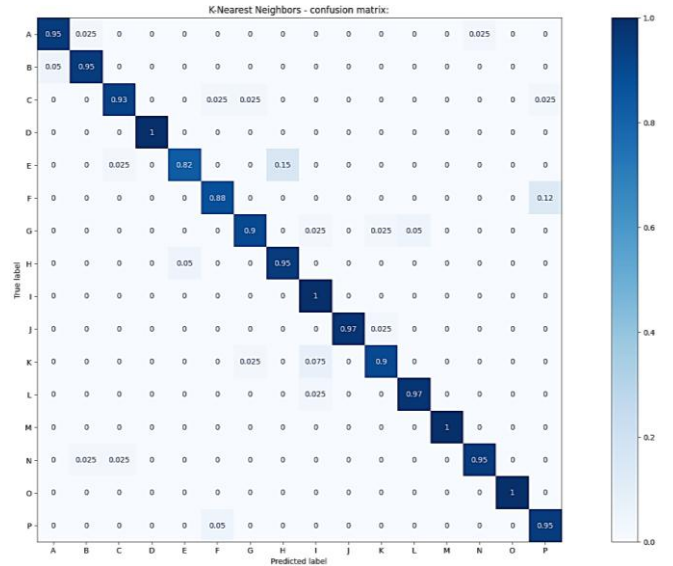


Fig. 5. Scenario A – KNN model confusion matrix

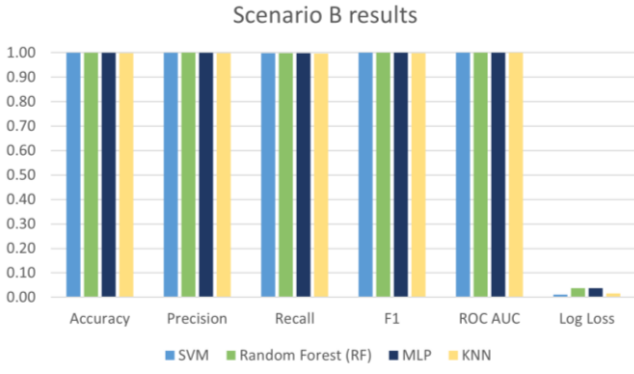


Fig. 6. Scenario B results chart

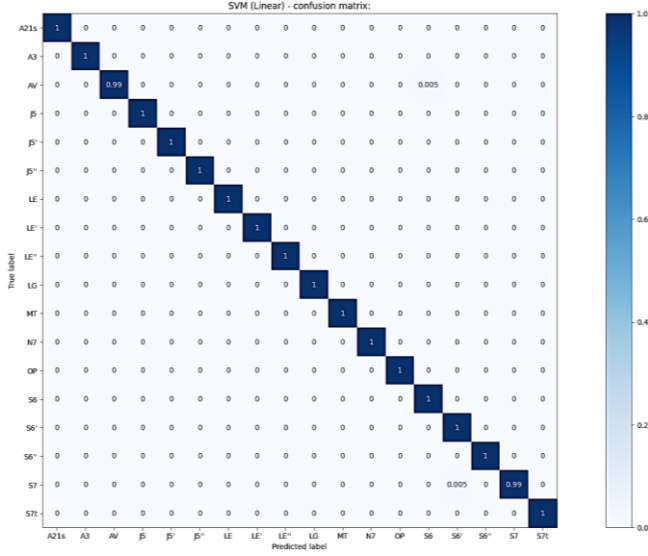


Fig. 7. Scenario B – SVM model confusion matrix

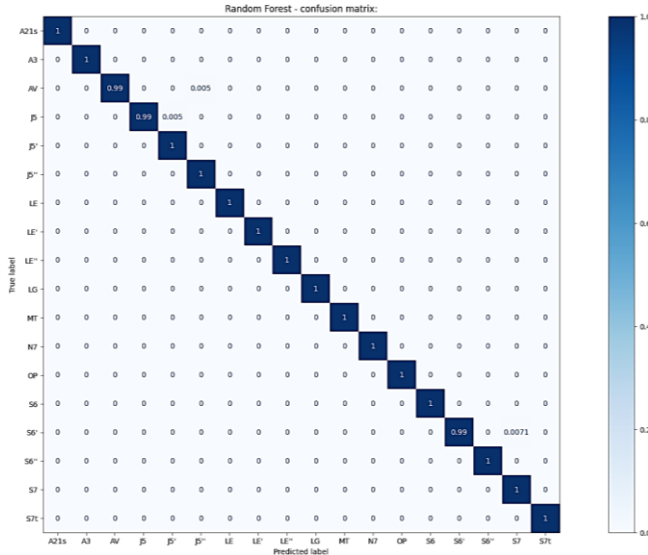


Fig. 8. Scenario B – RF model confusion matrix

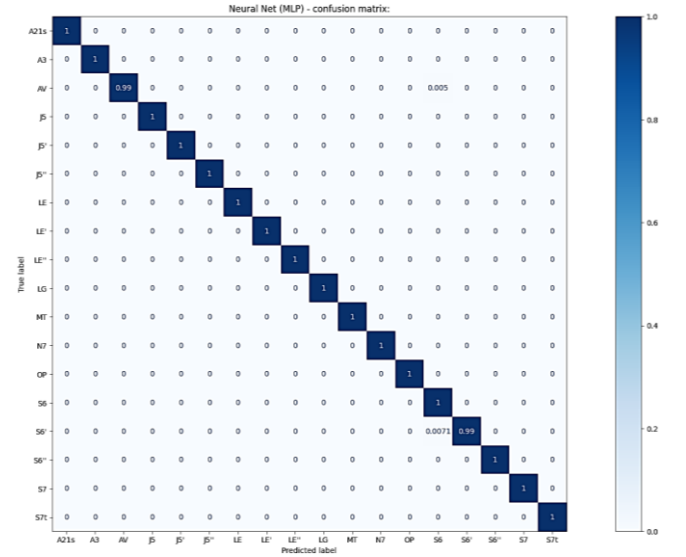


Fig. 9. Scenario B – MLP model confusion matrix

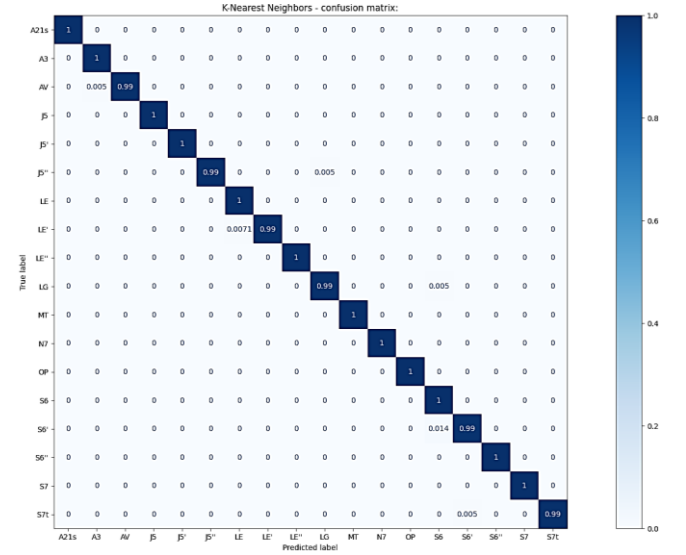


Fig. 10. Scenario B – KNN model confusion matrix

C. Scenario C

This scenario evaluates the performance of the four models when tested in a noisy environment, similar to real-life conditions. The recordings used are based on the same types of sounds from scenario B but recorded in an environment with background noise, i.e. music.

As this scenario provides the most difficult classification task of all three, the results for all models indicate that all four perform noticeably worse, as seen in Fig. 11, with KNN displaying the worst performance, with an accuracy just slightly better than a random guess and a very large log loss, indicating a high degree of confident misclassifications. However, the Random Forest model manages to achieve a good accuracy and good performance metrics generally, suggesting that an ensemble-based model, such as Random Forest, is capable of filtering out the environmental noise and focus on the intrinsic features of the devices microphones. The MLP model also comes close to the previous model's performance, indicating that artificial neural networks also possess a high degree of ability in distinguishing the features of this dataset, thus being a comparable alternative to the

Random Forest approach. The confusion matrices for the four models are presented in Fig. 12, Fig. 13, Fig. 14 and Fig. 15.

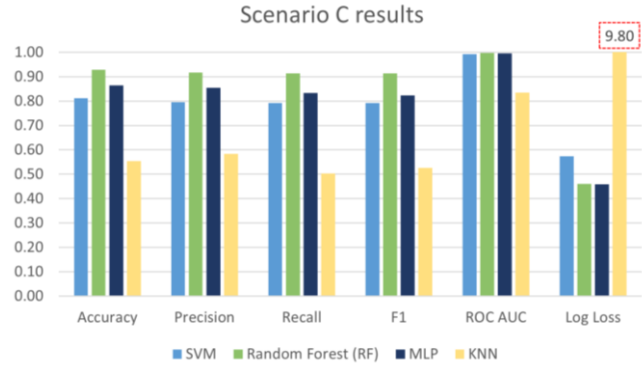


Fig. 11. Scenario C results chart

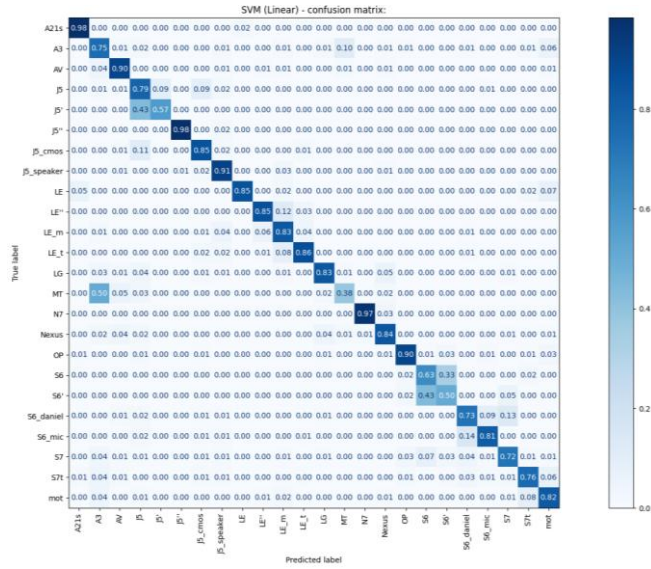


Fig. 12. Scenario C – SVM model confusion matrix

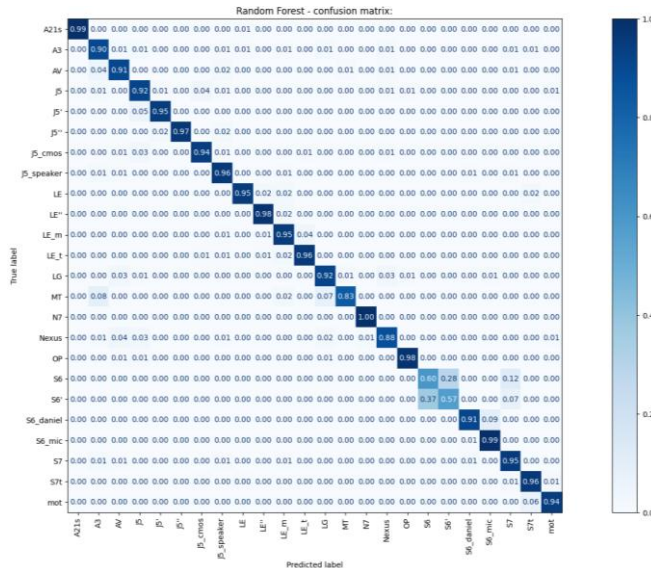


Fig. 13. Scenario C – RF model confusion matrix

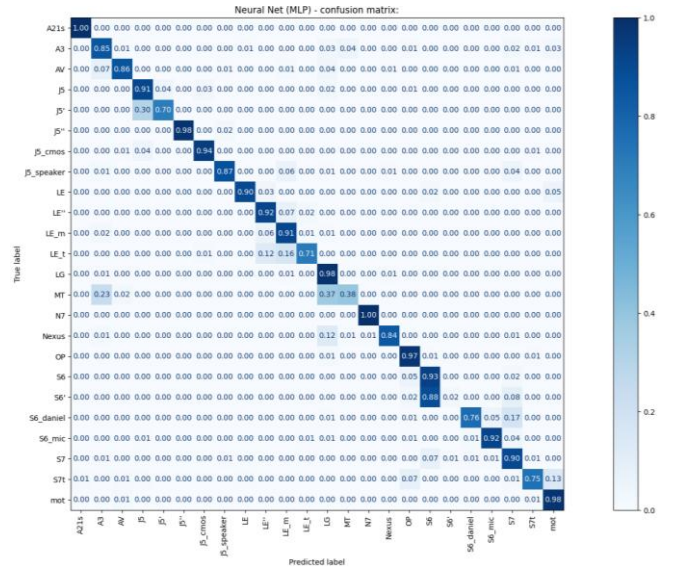


Fig. 14. Scenario C – MLP model confusion matrix

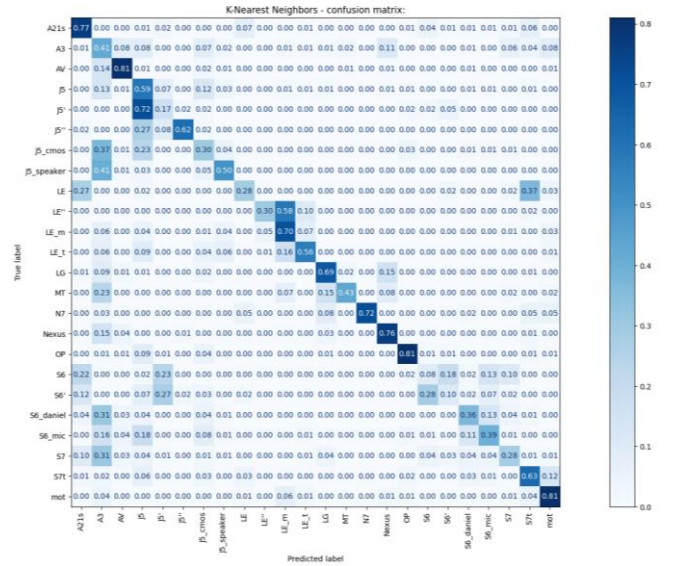


Fig. 15. Scenario C – KNN model confusion matrix

V. CONCLUSION

In conclusion, this paper discusses the performance of four classes of machine learning algorithms – namely *linear*, *ensemble-based*, *connectionist*, and *instance-based*, with a relevant algorithm from each class used – in the classification of microphone recording data for the purposes of device fingerprinting based on microphone recording characteristics. Three types of scenarios were taken into account: fingerprinting various units of the same device model based on synthetic clean recordings, and fingerprinting distinct models from various manufacturers using both clean and noisy recordings, with the latter scenario able to be described as being convergent with real-world usage.

The results indicate that, while in the first two scenarios all four models performed similarly well, the last test scenario created a more difficult condition for accurate classifications. As such, only the ensemble-based Random Forest model and the connectionist-based Multi-Layered Perceptron (MLP) model performed at a reasonable standard.

REFERENCES

- [1] J. Zhang, A. R. Beresford and I. Sheret, "Factory Calibration Fingerprinting of Sensors," in *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1626-1639, 2021, doi: 10.1109/TIFS.2020.3039685.
- [2] A. Das, N. Borisov, and M. Caesar, 'Do You Hear What I Hear? Fingerprinting Smart Devices Through Embedded Acoustic Components', in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, Scottsdale, Arizona, USA, 2014, pp. 441–452.
- [3] A. Berdich, B. Groza, E. Levy, A. Shabtai, Y. Elovici and R. Mayrhofer, "Fingerprinting Smartphones Based on Microphone Characteristics From Environment Affected Recordings," in *IEEE Access*, vol. 10, pp. 122399-122413, 2022, doi: 10.1109/ACCESS.2022.3223375.
- [4] Hsu, Chang, and Lin, *A practical guide to support vector classification*. 2003. [Online]. Available: https://eecs.csuohio.edu/~sschung/DSA460/SVM_guide.pdf
- [5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/a:1010933404324.
- [6] H. Ramchoun, M. Amine, J. Idrissi, Y. Ghanou, and M. Ettaouil, "Multilayer Perceptron: architecture optimization and training," *International Journal of Interactive Multimedia and Artificial Intelligence*, vol. 4, no. 1, p. 26, Jan. 2016, doi: 10.9781/ijimai.2016.415.
- [7] G. Guo, H. Wang, D. Bell, Y. Bi, and K. Greer, "KNN Model-Based Approach in Classification," in *Lecture notes in computer science*, 2003, pp. 986–996. doi: 10.1007/978-3-540-39964-3_62.